TITAN: A High-Throughput Distributed System for Approximate Nearest Neighbor Search on Large-scale Datasets

ZIYI LI*, Wuhan University, China
ANGJUN LIN*, Wuhan University, China
YUHAO LIN, Wuhan University, China
QUANQING XU, Ant Group, China
RUIXUAN ZHANG, Wuhan University, China
YUANYUAN ZHU, Wuhan University, China
JIAWEI JIANG, Wuhan University, China
XIAO YAN, Wuhan University, China

Approximate nearest neighbor search (ANNS) underlies many applications such as information retrieval, recommendation systems, and retrieval-augmented generation. As datasets scale to millions and even billions of vectors, distributed ANNS becomes essential but faces fundamental challenges due to excessive worker access and wasted local computation. To tackle these challenges, we propose TITAN, a distributed vector search system that co-designs data partitioning, query routing, and local search. The key designs include: (1) a two-tier architecture in which a global meta-index routes each query to a small set of workers, whose local sub-indexes on each worker conduct local search, (2) a graph-topology-aware overlapping partitioning strategy and an angular-based bucket pruning algorithm to reduce worker accesses, and (3) a hybrid sub-index that adaptively selects between searching the IVF and proximity graph indexes to reduce per-worker computation. Evaluation on 5 diverse datasets, together with an additional billion-scale evaluation on SIFT1B shows that TITAN improves query throughput by up to 4.52× over 6 SOTA baselines and achieves near-linear scalability.

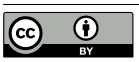
CCS Concepts: • **Information systems** → **Distributed retrieval**; *Information retrieval*.

Additional Key Words and Phrases: Approximate Nearest Neighbor Search, Distributed System, Inverted File Index, Proximity Graph Index

ACM Reference Format:
Ziyi Li, Angjun Lin, Yuhao Lin, Quanqing Xu, Ruixuan Zhang, Yuanyuan Zhu, Jiawei Jiang, and Xiao Yan. 2026. TITAN: A High-Throughput Distributed System for Approximate Nearest Neighbor Search on Large-scale Datasets. *Proc. ACM Manag. Data* 4, 6, Article 381 (December 2026), 27 pages. https://doi.org/10.1145/3856376

*Both authors contributed equally to this research.

Authors' Contact Information: Ziyi Li, School of Cyber Science and Engineering, Wuhan University, Wuhan, China, lzy0323@whu.edu.cn; Angjun Lin, School of Computer Science, Wuhan University, Wuhan, China, adrianj@whu.edu.cn; Yuhao Lin, School of Computer Science, Wuhan University, Wuhan, China, yuhao_lin@whu.edu.cn; Quanqing Xu, OceanBase, Ant Group, Hangzhou, China, xuquanqing.xqq@oceanbase.com; Ruixuan Zhang, School of Computer Science, Wuhan University, Wuhan, China, zrx123@whu.edu.cn; Yuanyuan Zhu, School of Computer Science, Wuhan University, Wuhan, China, yyzhu@whu.edu.cn; Jiawei Jiang, School of Computer Science, Wuhan University, Wuhan, China, jiawei.jiang@whu.edu.cn; Xiao Yan, Institute for Math & AI, Wuhan University, Wuhan, China, yanxiaosunny@whu.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2836-6573/2026/12-ART381
https://doi.org/10.1145/3856376

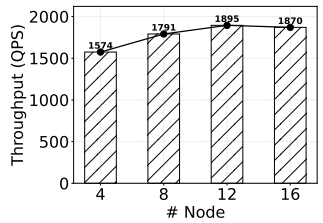


Fig. 1. Throughput scalability of Weaviate on SIFT100M dataset at 0.90 Recall@20 with worker nodes $\in [4, 8, 12, 16]$.

1 Introduction

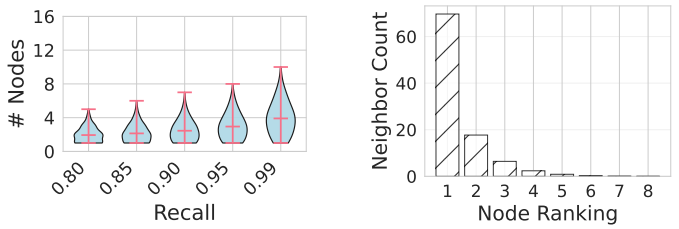
Nearest neighbor search (NNS) is a key component of many applications [11, 35, 36], including information retrieval [39, 47], recommendation systems [16, 30, 48] and retrieval-augmented generation (RAG) [1, 25, 49]. These applications encode various data modalities (e.g., texts, images, videos, and other unstructured data) into high-dimensional vectors using embedding models. Given a query vector, NNS retrieves the top-k most similar vectors from a base dataset, as measured by Euclidean distance, inner product, or cosine similarity. Since exact NNS requires a linear scan in high-dimensional spaces, *approximate nearest neighbor search (ANNS)* is widely utilized, trading marginal result accuracy for substantial query efficiency improvements.

Among the indexes for ANNS [8, 17], IVF [5, 46, 50] and proximity graph [9, 19, 28] are the most popular. In particular, IVF groups the vectors into clusters and represents each cluster with a center. A query first probes the centers and then scans the vectors in the nearest clusters. Although there are many variants, the key idea of the proximity graph is to organize the vectors as a graph by connecting similar vectors and to process each query via a beam search on the graph. Specifically, the beam search starts at an entry vector, computes neighbor distances when visiting a vector, and moves to the nearest but unvisited vector each time. Proximity graph requires fewer distance computations than IVF to achieve the same recall for ANNS, while IVF has a smaller memory footprint.

Challenges of distributed ANNS. Production vector datasets may contain millions and even billions of high-dimensional vectors [5, 19]. Applications can require a low query latency (e.g., sub-10ms for user-facing services) and a high search recall for good user experience [14], along with high query throughput to serve a large number of users. Hence, distributed ANNS with multiple workers is necessary to scale up both memory capacity (for storing large datasets) and processing power (for high query throughput).

Distributed ANNS systems partition the vector dataset X_b over N workers, where each worker maintains a local index over its assigned subset of vectors S_i such that $\bigcup_{i=1}^{N} S_i = X_b$. As such, distributed ANNS involves three key aspects: (i) *data partitioning* divides the dataset into partitions across the workers, (ii) *query routing* determines which partitions to access for each query, and (iii) *local search* runs on the selected workers to find candidate neighbors within each partition. Merging the local candidate neighbors from the workers for final search results is straightforward.

Challenge I: Excessive worker access due to inefficient data partitioning and query routing. Most industrial-grade vector databases (e.g., Milvus [13, 37], Weaviate [7], OceanBase [52]) adopt *uniform partitioning* and *broadcast routing*. In particular, vectors are assigned to workers based on hash or range partitioning over their IDs, and each query is routed to all workers for processing. We evaluate Weaviate with local HNSW index on each worker, and Figure 1 shows that its query throughput only improves marginally when scaling from 4 to 12 workers and even degrades slightly



(a) Node access count vs. recall (b) Neighbor count at 0.95 recall

Fig. 2. Node access characteristics analysis of similarity-based partitioning systems on SIFT100M dataset at k=100, taking Pyramid as an example.

with 16 workers. This is inefficient for distributed systems since throughput is expected to increase (almost) linearly with worker count.

The reason is that ANNS workload is difficult to partition. Relational queries usually target specific key or value ranges and thus can be cleanly partitioned via hash or range functions. However, partitioning datasets over workers makes it difficult to fully utilize the pruning power of vector indexes. Theoretically, the compute complexity of searching a vector dataset is $\log n$ with n being the dataset cardinality [28]. Searching all N workers yields a complexity of $N \log n/N$, which is close to $N \log n$. Intuitively, vector indexes prune distance computations to accelerate query processing. A vector that is not a promising candidate for global top-k neighbors on the entire dataset may become a good candidate for local top-k neighbors on each worker. As a consequence, computation pruning is less effective on the partitioned sub-datasets.

To avoid accessing all workers for each query, systems like Pyramid adopt *similarity-based partitioning and routing*. In particular, similar vectors are assigned to each worker (e.g., via k-means clustering), and a small meta-index is built by sampling the vectors in the dataset. A query first searches the meta-index to find the partitions (i.e., workers) that are likely to contain its neighbors and is then routed to these workers. However, as shown in Figure 2 (a), achieving 0.95–0.99 recall still requires Pyramid to access up to 8 out of the 16 workers. The reason is that the neighbors of a query scatter over multiple workers if the query is close to partition boundaries, and many vectors lie near partition boundaries, which are prevalent in high-dimensional spaces.

Challenge II: Wasted computation due to identical local search strategy across different workers. The aforementioned systems conduct identical local index search over the selected workers for each query. Proximity graph is usually utilized as the local index due to its high efficiency. This may be an overkill and wastes computation as shown in Figure 2 (b). Specifically, under the similarity-based partitioning of Pyramid, more than 70% of the ground-truth neighbors of each query are concentrated on the most prominent worker, while other workers contain far fewer ground-truth neighbors. As such, there is a chance to utilize fewer distance computations on the less prominent workers to reduce overall query computation.

Our solution TITAN. By tackling the excessive worker access and wasted computation on each worker for distributed ANNS, we propose a system dubbed **t**iered **i**ndex for high **t**hroughput **ANN**S (i.e., **TITAN**). TITAN uses a holistic co-design of data partitioning, query routing, and local search. The key insight is to maintain a global view of the entire dataset for routing, and spend different local search effort on each accessed worker according to the query.

In particular, TITAN adopts a *two-tier index architecture* that tightly integrates global routing with local search. During index construction, TITAN groups vectors in the base dataset into fine-grained

small buckets via hierarchical k-means-based clustering (cf. Section 5.1). A proximity graph is built on the bucket centroids as the *meta-index*, which serves as a global IVF index on the entire dataset. The base dataset is assigned to the workers in a similarity-based fashion like Pyramid, and k-means is conducted on the bucket centroids for worker assignment such that each worker holds similar buckets. Each worker builds a proximity graph index on its assigned vectors as the *sub-index* for local search. A query first searches the meta-index to identify its nearest bucket centroids and then is routed only to workers that store the corresponding buckets. This architecture exploits the spatial locality of vectors and accesses fewer than 16% of the total workers for each query (cf. Section 6.3), outperforming both uniform partitioning and Pyramid.

To reduce query computation during routing, we introduce two complementary optimizations. First, a *graph-topology-aware overlapping partitioning* reduces worker access by selectively replicating boundary buckets. Instead of uniform replication, our algorithm analyzes the meta-index neighborhood structure: buckets whose neighbors predominantly belong to other partitions are replicated to those partitions. Second, an *angular-based bucket pruning algorithm* filters low-contribution buckets before local search by comparing angular volumes computed from bucket centroids and radii, reducing the worker access while maintaining query recall.

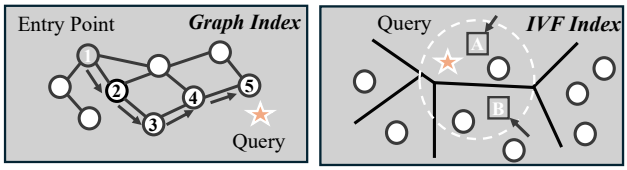
To avoid a full-fledged proximity graph search on the less prominent workers for each query, TITAN employs a *hybrid sub-index structure* locally on each worker. In particular, a worker maintains two views of its assigned vectors, i.e., as global IVF buckets and local proximity graph, and both indexes share the same full-precision vectors without the memory overhead of duplication. We obtain a bucket count threshold through profiling, below which scanning the IVF buckets is more efficient than a full-fledged proximity graph search. The search strategy on each worker (i.e., IVF or proximity graph) is determined by the number of buckets required by the query the worker stores. Moreover, TITAN also leverages an *entry optimization* to accelerate the proximity graph traversal on workers. Specifically, vanilla graph traversal starts from random or fixed entry points and requires a long traversal path for query processing. TITAN starts the local traversal from the nearest bucket centroid on the worker identified during the meta-index search. This effectively reduces the length of traversal path, as we utilize fine-grained small buckets and the nearest bucket is close to the query's neighbors.

To evaluate TITAN, we conduct extensive experiments on 5 datasets and compare TITAN with 6 baseline systems, including Sharding-based HNSW, Harmony [41], HAKES [15], Pyramid [6], Weaviate [40], Milvus [37] (cf. Section 6.1). We further conduct a dedicated billion-scale evaluation on SIFT1B. The results show that TITAN consistently outperforms all baselines across different datasets and recall targets. For 0.95 Recall@k=20, TITAN achieves a 1.53×–4.52× speedup over the runner-up system. Moreover, TITAN also exhibits good scalability, and its query throughput increases almost linearly with worker count. Profiling and ablation studies also confirm the effectiveness of our designs compared to alternatives.

To summarize, we make the following main contributions.

- We identify two fundamental challenges in distributed ANNS and propose TITAN[1] to tackle these challenges by co-designing data partitioning, query routing, and local search.
- We design a two-tier index structure for TITAN that integrates a replicated meta-index with a local sub-index on each worker node, reducing the node access to 16% of workers on average and addressing the limitation of inefficient data partition and routing.
- We design a graph-topology-aware overlapping partitioning that adaptively replicates boundary buckets based on the meta-index and an angular-based bucket pruning algorithm to filter low-contribution buckets, thereby further improving query efficiency.

[1]https://github.com/WHU-STAR/TITAN



(a) Proximity Graph Index (b) IVF Index

Fig. 3. An illustration of Proximity Graph Index and IVF Index. (a) Proximity Graph Index performs traversal-based search from entry points. (b) IVF Index performs bucket-based search by scanning selected buckets.

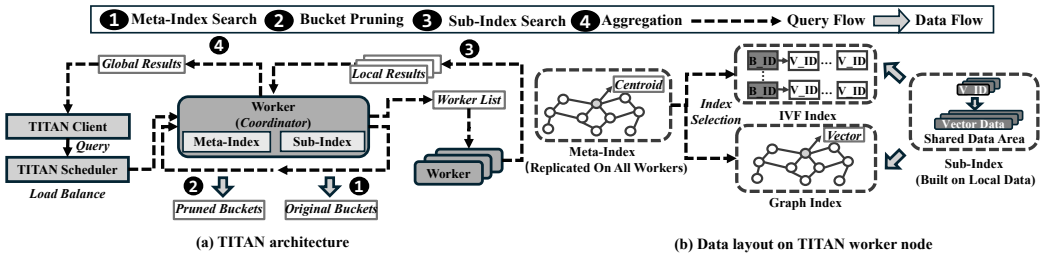


Fig. 4. Overview of TITAN's architecture and the data layout on a single worker node.

- We design a hybrid sub-index structure that adaptively selects between IVF and proximity graph index, combined with entry optimization to accelerate graph traversal, reducing distance computations by 55%–79% compared to uniform search strategies.

2 Preliminaries

Approximate Nearest Neighbor Search. Let $\mathcal{D}$ denote a dataset of n vectors embedded in a d-dimensional space $\mathbb{R}^d$. Given a query vector $\mathbf{x}$, the dissimilarity (or similarity) between $\mathbf{x}$ and any data vector $\mathbf{v} \in \mathcal{D}$ is measured by a distance (similarity) function $d(\mathbf{x}, \mathbf{v})$. Common choices for $d(\cdot, \cdot)$ include the Euclidean ℓ_2 distance, inner product, and cosine similarity. The exact k-nearest neighbor (k-NN) set of $\mathbf{x}$, denoted by $\mathcal{R} \subseteq \mathcal{D}$ with $|\mathcal{R}| = k$, contains the k vectors in $\mathcal{D}$ that have the smallest distances to $\mathbf{x}$, i.e.,

$$\mathbf{v} \in \mathcal{R} \implies \forall \mathbf{u} \in \mathcal{D} \setminus \mathcal{R},\ d(\mathbf{x}, \mathbf{v}) \leq d(\mathbf{x}, \mathbf{u})$$

However, identifying $\mathcal{R}$ exactly in high-dimensional spaces is computationally expensive due to the *curse of dimensionality* [18]. To overcome this limitation, modern vector search systems typically adopt *approximate nearest neighbor search* (ANNS) methods, which construct specialized index structures to efficiently retrieve a candidate set $\mathcal{R}'$ that approximates the exact nearest neighbor set $\mathcal{R}$ with substantially reduced computational cost. The effectiveness of an ANN method is commonly evaluated by its *Recall*, defined as

$$\text{Recall} = |\mathcal{R} \cap \mathcal{R}'| / |\mathcal{R}| \tag{1}$$

while its efficiency is measured by the query throughput in *queries per second* (QPS). Together, both metrics reflect the trade-off between retrieval accuracy and system efficiency in ANNS.

Proximity Graph Index. Proximity graph indexes, such as NSW [27], HNSW [28] and NSG [9], organize vectors into a proximity graph, where each vertex represents a vector and edges connect pairs of vectors that are mutually similar. As illustrated in Figure 3 (a), the graph traversal begins

from one entry point, explores the neighbors when visiting each vector, and progressively moves to the top-k neighbors of the query by visiting the most similar vector each time. This process continues until no closer vectors can be found.

The Hierarchical Navigable Small World (HNSW) algorithm [28] is a widely adopted graph-based approach to approximate nearest neighbor search. It extends the basic Navigable Small World (NSW) by introducing a multi-layer hierarchical structure. The bottom layer of HNSW contains all vectors, while upper layers are progressively sparser samples. Queries start from the sparsest top layer and descend to denser layers for finer-grained search. This design provides logarithmic scaling of traversal steps while maintaining high recall, making HNSW the de facto graph index in modern vector databases such as FAISS [20], Milvus [37] and Weaviate [40].

IVF Index. The Inverted File Index (IVF) is another widely adopted indexing method for large-scale ANNS. It leverages clustering-based partitioning to reduce the search space and accelerate similarity computations. Specifically, the dataset is partitioned into N_c disjoint *buckets*, often referred to as *inverted lists*, and typically obtained through a k-means clustering. Each bucket contains a set of spatially similar vectors. Bucket centroids are learned from a representative subset of the dataset and serve as reference anchors for assigning the remaining vectors to their nearest buckets based on distance or similarity metrics. As illustrated in Figure 3 (b), a query vector is first compared to all centroids of buckets (e.g., A, B) to identify the most relevant region. The subsequent search is then restricted to the top n_{probe} nearest buckets, within which candidate vectors are then exhaustively compared against the query vector. This selective probing reduces the number of distance computations and yields a substantial gain in efficiency. The parameters N_c and n_{probe} jointly control the balance between accuracy and efficiency using the following rules: a larger N_c improves partition granularity but increases centroid comparison costs, whereas a higher n_{probe} improves recall at the expense of additional computations.

Distributed ANNS Execution Model. In a distributed vector search system, the dataset is partitioned across multiple workers. Each *worker* stores a subset of the vectors (referred to as a *partition*) and maintains a local ANN index over the subset. Given a query vector, the system must determine which workers to access, execute local searches on the selected workers, and merge their local candidates into the final global top-k result. These three stages during query can be referred to as *query routing*, *local search* and *result aggregation*, respectively. We further distinguish three roles in this model. A *scheduler* assigns each incoming query to a worker according to the current system load. *A coordinator* determines which workers to send the query, dispatches local search requests to these workers and aggregates their returned candidates. A *worker* stores its partition and executes local ANN searches. Section 3 details how our system instantiates these roles.

3 System Architecture

In this section, we present the architecture and end-to-end query workflow of TITAN. Section 4 then describes its optimization techniques along two dimensions: (1) reducing worker accesses and (2) reducing computation on each accessed worker.

3.1 Overview

System Roles. Figure 4 (a) illustrates the architecture of TITAN. We instantiate the roles of Section 2 as follows. The scheduler assigns each incoming query to a worker according to system load. The selected worker temporarily acts as the coordinator for that query. It performs query routing, dispatches local search requests to relevant workers, and aggregates their results. Since

any worker can act as the coordinator and all expensive searches are executed by workers, the scheduler stays lightweight and does not become a centralized processing bottleneck.

Data Layout. Figure 4 (b) shows the data layout on a single worker. Following Section 2, TITAN organizes vectors into buckets by clustering. Each worker stores a partition, i.e., a subset of buckets. TITAN adopts a two-tier index consisting of a replicated *meta-index* and a local *sub-index*. The meta-index is a proximity graph over bucket centroids, giving each worker a compact global view for query routing. The sub-index provides two access paths over the same local vectors: (i) an IVF index for scanning specified buckets and (ii) a proximity graph for traversal-based search. Both indexes share the underlying vectors, avoiding local data duplication.

Query Workflow. As shown in Figure 4, each query is processed in four stages: meta-index search (Step ❶), which identifies the k_{meta} nearest buckets; bucket pruning (Step ❷), which filters out low-contribution buckets using angular heuristics; sub-index search (Step ❸), which adaptively chooses between IVF and graph index on a per-worker basis; and result aggregation (Step ❹), which merges local top-k candidates into the final global top-k result. The first two steps reduce the number of accessed workers, while the third step reduces wasted computation on each accessed worker.

Theoretical Analysis. We analyze how reducing worker accesses and per-worker search cost jointly improves the per-query computation and complexity of TITAN. In a system with N workers, n vectors and M buckets of average size $B = n/M$, a broadcast-based method dispatches every query to all N workers, incurring a total cost of $O(N \cdot \log(n/N) \cdot C_{\text{deg}})$, where C_{deg} is the average graph degree. In contrast, TITAN first searches the meta-index in $O(\log M \cdot C_{\text{deg}})$ time to identify k_{meta} nearest bucket centroids, then routes the query to only $\eta = |\mathcal{W}_q| \leq \min(k_{\text{meta}}, N)$ workers. The high quality meta-index ensures that $\eta \ll N$, so that the k_{meta} retrieved centroids are tightly concentrated in a small number of partitions. Therefore, the total cost of TITAN is $T_{\text{TITAN}} = O\big(\eta \cdot \log(k_{\text{meta}} \cdot B) \cdot C_{\text{deg}}\big)$, where the meta-index search overhead $O(\log M \cdot C_{\text{deg}})$ is negligible in comparison. The theoretical speedup over the baselines is thus:

$$\frac{T_{\text{broadcast}}}{T_{\text{TITAN}}} = \frac{N}{\eta} \cdot \frac{\log(n/N)}{\log(k_{\text{meta}} \cdot B)}. \tag{2}$$

The two factors correspond to TITAN's two optimization dimensions: N/η captures the reduction in accessed workers, while the other factor $\log(n/N)/\log(k_{\text{meta}} \cdot B)$ captures the reduction in the search space on each accessed worker. Since $\eta \ll N$ and $k_{\text{meta}} \cdot B \ll n/N$ in practice, both factors substantially reduce the per-query computation. The adaptive local-search techniques in Section 4.2 further decrease the per-worker cost.

4 Optimization Techniques

Following the cost decomposition above, we organize TITAN's techniques into two groups: reducing the number of workers accessed by each query (Section 4.1) and reducing the computation performed on each accessed worker (Section 4.2).

4.1 Reducing Worker Accesses

Graph-aware Overlapping Partitioning. Disjoint partitioning minimizes storage overhead but may require accessing multiple workers when a query lies near a partition boundary. Uniform

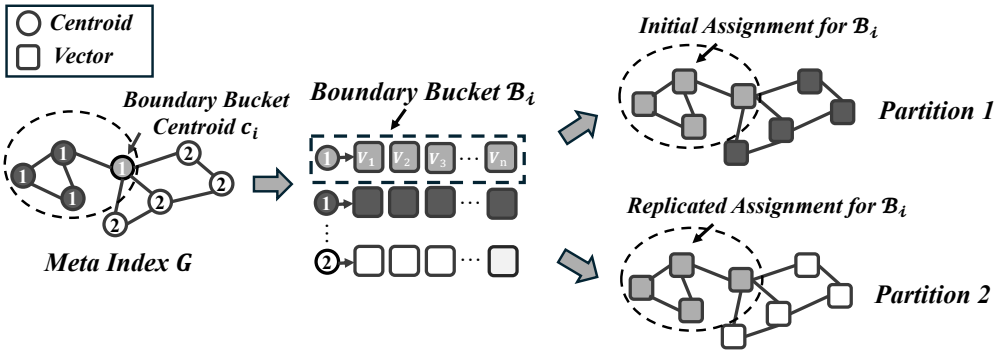


Fig. 5. An illustration of graph-aware overlapping partitioning. Since bucket $\mathcal{B}_i$'s centroid c_i lies near the partition boundary of meta index graph G, it is replicated from its original assignment (Partition 1) to Partition 2.

Algorithm 1 Graph-aware Overlapping Partitioning

Input: Number of buckets M, number of partitions N;
Bucket centroids $C = \{c_1, \ldots, c_M\}$, Meta Index G over C;
Parameters k, $\theta \in (0, 1)$, $n_{\max} \leq N$
Output: Partition assignment $A : \{1, \ldots, M\} \to 2^{\{1,\ldots,N\}}$

// Phase 1: Compute partition centroids and initial assignment
$\{\mu_1, \ldots, \mu_N\} \leftarrow k$-means (C, N)
for $i = 1$ to M **do**
    $\pi(\mathcal{B}_i) \leftarrow \arg\min_j \|c_i - \mu_j\|_2$
    $A(i) \leftarrow \{\pi(\mathcal{B}_i)\}$
// Phase 2: Selective replication based on graph topology
for $i = 1$ to M **do**
    // Get k approximate nearest neighbors
    $\mathcal{N}_i \leftarrow \text{ANNQuery}(G, c_i, k)$
    for $j = 1$ to N **do**
        // Compute the partition distribution of the neighbors
        $\rho_i(j) = |\{\mathcal{B} \in \mathcal{N}_i : \pi(\mathcal{B}) = j\}|/|\mathcal{N}_i|$
    Sort $j_1, j_2, \ldots$ by descending $\rho_i(\cdot)$
    for each $j \in \{j_1, j_2, \ldots\}$ **and** $|A(i)| < n_{\max}$ **do**
        if $j \neq \pi(\mathcal{B}_i)$ **and** $\rho_i(j) \geq \theta$ **then**
            $A(i) \leftarrow A(i) \cup \{j\}$ ▷ *Replicate bucket* $\mathcal{B}_i$
return A

overlapping partitioning reduces such accesses by assigning every bucket to $n_{\max}$ nearby partitions, but incurs up to $n_{\max}\times$ storage overhead regardless of whether replication is useful.

Design Intuition. Buckets have different boundary characteristics. A bucket whose meta-index neighbors mostly fall in one partition is likely inside that partition, whereas a bucket whose neighbors span several partitions is likely near a boundary. Queries near the latter often need vectors from multiple partitions. We call such a bucket a *boundary bucket* and selectively replicate it to the neighboring partitions indicated by the meta-index topology. As illustrated in Figure 5,

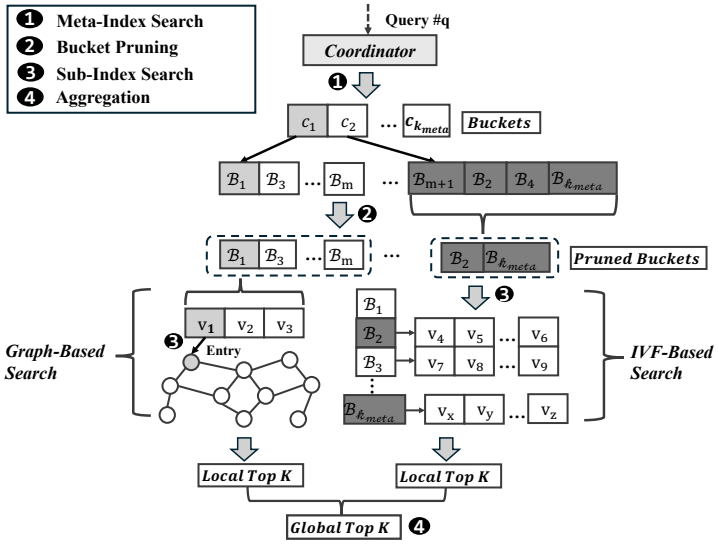


Fig. 6. Detailed query workflow of TITAN.

bucket $\mathcal{B}_i$ is initially assigned to Partition 1, but its centroid is predominantly adjacent to centroids belonging to Partition 2. TITAN therefore replicates $\mathcal{B}_i$ to Partition 2, allowing nearby queries to cover the relevant vectors with fewer worker accesses.

Detailed Process. Algorithm 1 consists of two phases: initial assignment and adaptive replication. In the first phase, we perform k-means clustering on the bucket centroids C to obtain N partition centroids $\{\mu_1, \ldots, \mu_N\}$, and each bucket $\mathcal{B}_i$ is assigned to its nearest partition centroids: $\pi(\mathcal{B}_i) = \arg\min_j \|c_i - \mu_j\|_2$ (Lines 3–5). For adaptive replication, TITAN retrieves the k approximate nearest neighbors $\mathcal{N}_i$ of each bucket centroid c_i from the meta-index (Line 9) and computes the fraction assigned to partition j as

$$\rho_i(j) = |\{\mathcal{B} \in \mathcal{N}_i : \pi(\mathcal{B}) = j\}| / |\mathcal{N}_i| \tag{3}$$

The algorithm then sorts the partitions by $\rho_i(j)$ in descending order and examines them sequentially (Lines 13–16). A partition $j \neq \pi(\mathcal{B}_i)$ is added to $\mathcal{B}_i$'s assignment if $\rho_i(j) \geq \theta$, indicating that a significant fraction of $\mathcal{B}_i$'s graph neighbors belong to partition j. The replication process terminates when either all qualifying partitions have been considered or the maximum assignment count $n_{\max}$ is reached, ensuring the upper bound of storage overhead.

This strategy enables adaptive replication, as buckets with homogeneous neighborhoods remain assigned to a single partition, while boundary buckets with diverse neighborhoods are replicated to more partitions based on their topological relationships. With overlapping partitioning enabled, the coordinator employs a greedy algorithm to select the minimum set of workers whose union of buckets covers the required buckets, thereby reducing node access.

Meta-Index Search. For a query vector q, the coordinator searches the replicated meta-index for the k_{meta} nearest bucket centroids, where $k_{meta} > k$ preserves sufficient candidates for high recall. Each returned bucket records the worker(s) storing its vectors, producing the query-specific worker set $\mathcal{W}_q \subseteq \{w_1, \ldots, w_N\}$. The coordinator therefore routes the query to only $|\mathcal{W}_q|$ workers instead

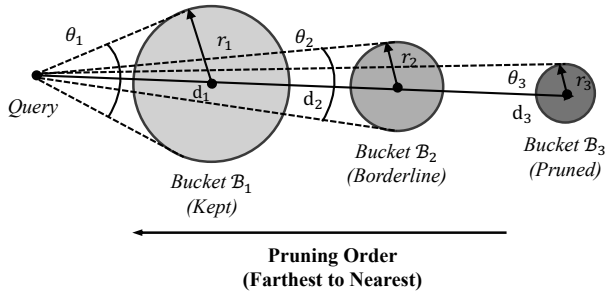


Fig. 7. Illustration of Angular-Based Bucket Pruning. Bucket $\mathcal{B}_1$ is kept with larger θ_1, while $\mathcal{B}_3$ is pruned.

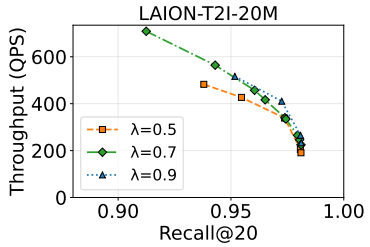


Fig. 8. Recall-QPS under different λ on LAION-T2I-20M.

of broadcasting to all N workers. Since the meta index contains only n_c centroids, its graph search takes $O(\log n_c)$ time.

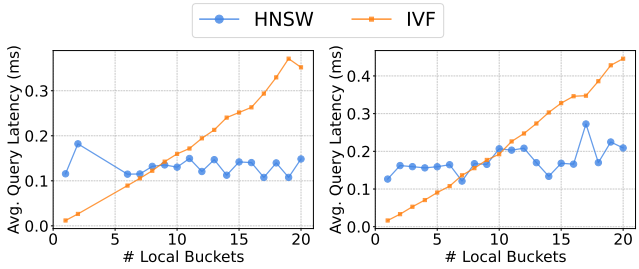
Algorithm 2 Angular-Based Bucket Pruning

Input: Candidate bucket list $B = \{\mathcal{B}_1, \ldots, \mathcal{B}_n\}$, with distances $\{d_1, \ldots, d_n\}$ and radii $\{r_1, \ldots, r_n\}$; Retention rate $\lambda \in [0, 1]$

Output: Pruned bucket set B'

$S \leftarrow 0$ ▷ *Sum of all volume factors*
for each bucket $\mathcal{B}_i \in B$ **do**
  $\theta_i \leftarrow \arctan(r_i/d_i)$ ▷ *Viewing angle*
  $f_i \leftarrow (\theta_i/(\pi/2))^2$ ▷ *Volume factor*
  $S \leftarrow S + f_i$
$L \leftarrow 0, L_{\max} \leftarrow 1 - \lambda, B' \leftarrow \emptyset$
for each bucket $\mathcal{B}_i \in B$ **from** farthest **to** nearest **do**
  $\alpha_i \leftarrow f_i/S$ ▷ *Normalized contribution*
  if $L + \alpha_i \le L_{\max}$ **then**
    $L \leftarrow L + \alpha_i$ ▷ *Prune bucket* $\mathcal{B}_i$
  else
    $B' \leftarrow B' \cup \{\mathcal{B}_i\}$ ▷ *Keep bucket* $\mathcal{B}_i$
return B'

Angular-Based Bucket Pruning. Before local search on sub-index, the coordinator applies a pruning step to filter buckets that contribute minimally to the final results (e.g., $\mathcal{B}_{m+1}$ and $\mathcal{B}_4$).



(a) Deep100M (b) SIFT100M

Fig. 9. Average query latency of HNSW and IVFFlat index on Deep100M and SIFT100M at 0.95 Recall@k=20. IVFFlat's latency scales linearly with the number of local buckets due to sequential scanning, whereas HNSW remains stable as graph traversal is independent of bucket boundaries.

Algorithm 2 details the process, which models each bucket's contribution through the solid angle it subtends at the query point. For a bucket at distance d_i with radius r_i (the maximum distance from its centroid to any contained point), the algorithm computes its viewing angle as $\theta_i = \arctan(r_i/d_i)$ (Line 3).

Geometric Intuition. We make a simplifying assumption that the vectors in a bucket follow a uniform distribution. θ_i captures the angular extent of the bucket from the query point. A larger θ_i indicates that the bucket subtends a larger solid angle from the query point and is more likely to contain nearest vectors. We treat θ_i only as a lightweight ranking signal, not as an exact containment probability. Figure 7 illustrates this geometrically in a 2-D schematic.

For computational convenience, we define a volume factor $f_i = (\theta_i/(\pi/2))^2$ to approximate the volumetric contribution of the bucket while avoiding high-dimensional computation (Line 4). The algorithm computes the normalized contribution of each bucket $\alpha_i = f_i/S$ with $S = \sum_j f_j$ (Line 8), ensuring $\sum_{i=1}^{n} \alpha_i = 1$. This lets the retention rate λ cap the pruned mass as a fraction of that total.

Contribution Maximization. Given a retention rate $\lambda \in [0, 1]$, the objective of the algorithm is to maximize the number of pruned buckets while ensuring that the total volumetric contribution of pruned buckets does not exceed the upper bound $1 - \lambda$. Scanning buckets from farthest to nearest, the algorithm prunes $\mathcal{B}_i$ if adding its contribution keeps the cumulative loss within $L + \alpha_i \leq 1 - \lambda$, otherwise keeps it in $\mathcal{B}'$ (Line 9-12). This greedy strategy maximizes the number of pruned buckets under the loss cap. Sorting runs in $O(n \log n)$ time and pruning runs in $O(n)$ time. The parameter λ provides an intuitive trade-off between search efficiency and accuracy, as lower λ prunes more aggressively for higher throughput and higher λ retains more buckets for better recall. We tune the parameter for evaluation in Section 6.3.

Robustness of Angular Pruning on Skewed Data. Real buckets can be non-uniform, so a single score can mis-rank an irregular cluster. TITAN uses a large number of fine-grained buckets, which limits the impact of such errors on the retained set. Figure 8 reports the results on the skewed LAION-T2I-20M dataset while varying λ. Even under aggressive pruning (λ=0.5), TITAN still reaches Recall>0.95, empirically confirming the robustness of the algorithm.

4.2 Reducing Per-Worker Computation

After routing and pruning, the selected workers receive very different numbers of relevant buckets. TITAN therefore adapts the local search strategy independently for each query–worker pair.

Index Selection via the Dual Index. As discussed in Challenge II, applying a uniform search strategy across all accessed workers leads to the wasted computation problem. To understand the root cause, we profile the query latency of IVFFlat and HNSW indexes under varying levels of bucket selectivity on queries. Specifically, we fix the total number of queries and vary the number of buckets they span, measuring the average query latency of each index. As shown in Figure 9, we observe two distinct behaviors. First, the latency of IVF is roughly proportional to the bucket count, because IVF sequentially scans the vectors within each specified bucket and aggregates the results, accessing fewer buckets means fewer scattered memory accesses and less overhead. Second, the latency of HNSW remains relatively stable regardless of the bucket count as normal graph traversal is independent of bucket boundaries. Consequently, IVF achieves significantly lower latency when the query targets only a small number of buckets (i.e., low selectivity), while HNSW is more efficient when the query spans many buckets. This observation motivates an adaptive index selection mechanism based on query selectivity.

Selection Mechanism. As illustrated in Figure 6, if the number of assigned bucket IDs $|\mathcal{B}_i|$ is below a threshold τ (indicating low selectivity), worker w_i uses the *IVF-based search* path by directly retrieving vectors from the specified buckets (e.g., $\mathcal{B}_2, \mathcal{B}_{k_{meta}}$) and computes their distances to q. Conversely, when $|\mathcal{B}_i| \geq \tau$, it switches to the *graph-based search* path by performing greedy graph traversal to avoid exhaustive scanning over a large number of buckets. We fix τ offline at the IVF/HNSW latency crossover on the validation set under recall target. If the bucket-count distribution drifts, the same profiling can be re-run without changing the dual-index design.

Entry Optimization. For graph-based search, traditional solutions start from a random point or a global centroid of the proximity graph. In contrast, TITAN leverages the results of meta-index search to initialize the traversal from optimal entry points. Specifically, for the query's nearest bucket centroid, we use the bucket's precomputed nearest point as the starting node. For example, in Figure 6 (Step ❸), c_1 is the centroid of the bucket closest to query q, and v_1 is the precomputed closest point to c_1, so it serves as the entry point to start with. This entry point is retrieved in $O(1)$ time using a precomputed mapping and significantly shortens the graph traversal path. Furthermore, this mechanism seamlessly bridges the two-tier index rather than constructing a single global graph over all points.

Finally, each worker returns its local top-k results to the coordinator. The coordinator merges all candidates and selects the global top-k by distance, then returns the results to the client (Step ❹).

4.3 Discussions

Update Handling. Although dynamic updates are outside the query-processing focus of this work, existing update paradigms can be easily integrated into TITAN: **(1) In-place Incremental Updates:** Algorithms like SPFresh [42] and Wolverine [24] support local online updates with limited synchronization. In SPFresh, foreground updates are decoupled from background rebalancing and posting reads are lock-free; centroids, and hence meta-index entries, change only upon structural rebalancing rather than every vector update. **(2) LSM-style Delta Merging:** Similar to Milvus [37], new vectors stream into a mutable delta index, which is queried concurrently and bulk-merged in the background. Both approaches strictly decouple foreground queries from background maintenance, avoiding stalling. Since index update is orthogonal to distributed ANNS, we leave the integration for future work.

Memory Overheads. Our fine-grained meta index is replicated on all workers, and the graph-topology-aware overlapping partitioning also replicates buckets over the workers. These memory

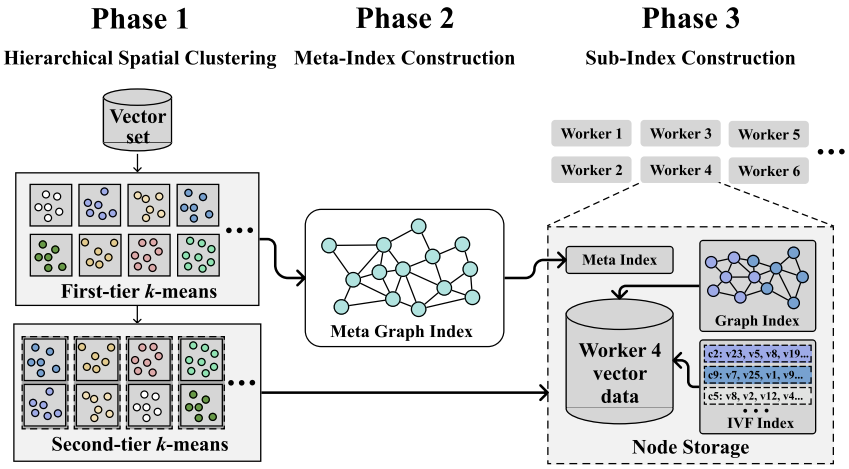


Fig. 10. The detailed workflow of index construction.

overheads can be reduced with vector quantization techniques like RaBitQ [10] and SAQ [23]. In particular, the meta-index, IVF index, and proximity graph index can store 6-8× compressed approximate vectors with only marginal degradation in distance accuracy. A single copy of the full-precision uncompressed vectors is stored across the workers and utilized only to re-rank the search results.

5 Implementation

We implemented TITAN using approximately 20K lines of code (LoC), combining C++ for performance-critical components and Go for concurrency management. The index construction is implemented by 3K LoC in Python. For the proximity graph index, we provide a self-implemented Vamana index and an enhanced HNSW index built with hnswlib [28]. For the IVF index, we use IVFFlat combined with SIMD optimization. TITAN can be readily deployed in general-purpose servers with standard hardware configurations.

5.1 Index Construction

We construct the indexes of TITAN following the procedure illustrated in Figure 10. The construction pipeline produces a two-tier indexing architecture consisting of a global meta-index for query routing and per-worker sub-indexes for local search.

Phase 1: Hierarchical Spatial Grouping. We first apply k-means to partition the dataset into M buckets with centroids, then assign them to N second-tier clusters following the algorithm described in Section 4.1. Each bucket produced in the first stage is assigned to up to $n_{\max}$ corresponding workers.

Phase 2: Meta-Index Construction. A global meta-index is then built over the bucket centroids of Phase 1, which is replicated on all worker nodes to enable decentralized query processing.

Phase 3: Sub-Index Construction. Each worker builds a sub-index (IVF + proximity graph) over a shared, single-copy vector storage, where base vectors are placed in a single contiguous array, with vectors belonging to the same bucket placed in contiguous memory. For each bucket, the IVF-based structure maintains an offset and length of its corresponding segment within this array.

Dataset	# Base Vectors	# Dimension	# Domain
SIFT100M	100,000,000	128	Image
Deep100M	100,000,000	96	Image
SpaceV100M	100,000,000	100	Text
LAION-T2I-20M	20,000,000	768	Text&Image
GIST1M	1,000,000	960	Image

Table 1. Datasets

This design enables $O(1)$-time lookup of bucket contents, which ensures high spatial locality during bucket-restricted search.

5.2 User-friendly APIs

The core interfaces of TITAN. We provide concise and intuitive API interfaces, similar to mainstream ANNS libraries such as FAISS [20]. The core interfaces of TITAN are as follows:

- `InitCluster(config)`: Initializes a TITAN cluster with user-specified configurations, including network configuration, index types, data partition strategies, and search parameters.
- `BuildIndex(n, x)`: Constructs the distributed two-tier index from `n` input vectors stored in array `x` across workers. The system automatically partitions the dataset, builds local sub-indexes on each worker, and replicates the meta-index to all workers via distribution. Both indexes can be persisted to disk for reuse.
- `Search(nq, q, k)`: Executes distributed nearest neighbor search for `nq` query vectors `q` in batch mode, returning the top-k results.

These interfaces provide a clean separation between index building and query processing. The simplicity of API hides the underlying implementation of techniques, allowing users to benefit from the full capabilities of TITAN without additional system-level tuning.

Unified Index Interfaces. To support diverse workload characteristics, TITAN allows independent configuration of index structures. We design the index module with unified APIs, allowing users to easily integrate new index structures or customize existing ones. Each index module implements the following three core interfaces:

- `Index.Load(path, dim, metric)`: Loads the pre-built index from disk with specified dimension and distance metric.
- `Index.Search(vector, k)`: Performs standard ANNS and returns the local top-k results with their distances.
- `Index.SearchWithEntry(vector, bucketId, k)`: Performs search starting from the entry point corresponding to `bucketId`, enabling entry optimization as described in Section 4.2.

All components conform to these interfaces, so indexes can be replaced without modifying the routing and aggregation logic. This provides flexibility as new indexes emerge, while maintaining consistent performance across different deployment scenarios.

6 Evaluation

We compare TITAN with state-of-the-art solutions using a cluster of 16 worker nodes to evaluate the effectiveness of our design.

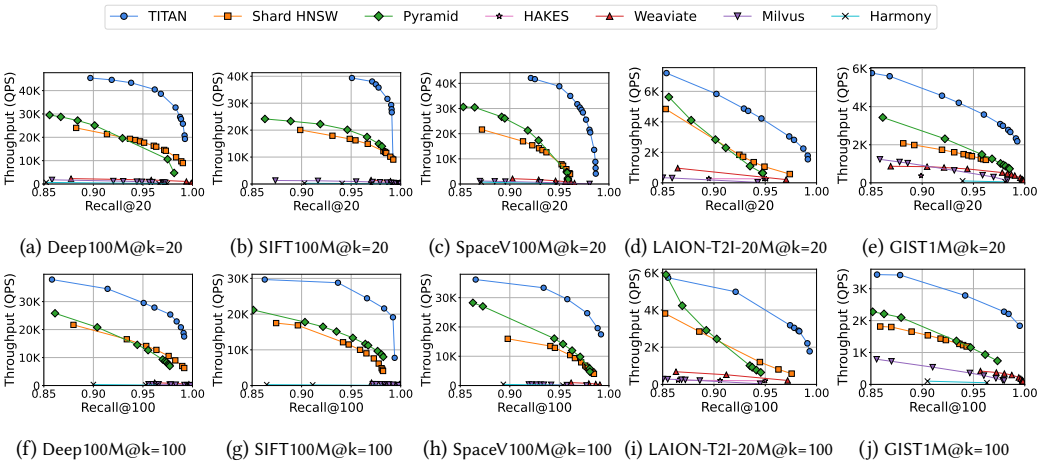


Fig. 11. The query recall-throughput curves of TITAN and the baselines.

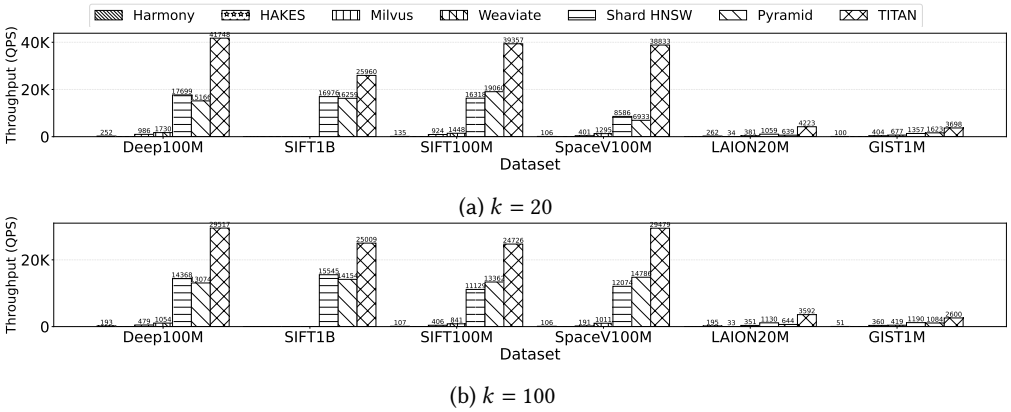


Fig. 12. The query throughput comparison of TITAN and the baselines at the 0.95 Recall@k target.

6.1 Experiment Settings

Datasets. We conduct our main experiments using five diverse datasets widely adopted in prior ANN studies and benchmarks: SIFT100M [22], Deep100M [43], LAION-T2I-20M [32] (sampled from LAION-5B dataset), and SpaceV100M [29] from BIG-ANN [34], and GIST1M [21], a 960-dimensional image-descriptor dataset used to evaluate high-dimensional search. Table 1 summarizes their key characteristics. To ensure a fair comparison with baselines that only support floating-point vectors and lack native int8 or uint8 support, we convert the original SIFT100M (uint8) and SpaceV100M (int8) vectors into float32 format. We additionally use the billion-scale SIFT1B dataset in the dedicated evaluation in Section 6.2.

Compared methods. We compare TITAN against six distributed vector search systems, covering both industrial-grade vector databases and specially designed systems.

- Shard HNSW represents the most naive distributed approach. It randomly partitions the dataset across all nodes and constructs an independent HNSW graph on each node, with queries broadcast to all nodes for parallel search. We implement Shard HNSW based on hnswlib as it is superior to FAISS-CPU [2].
- Pyramid [6] randomly samples vectors from base vector set to build a HNSW graph as route-index, which is partitioned into balanced shards using balance graph-partitioning algorithm [31]. With the route-index, queries are routed to the specific node. As the source code of Pyramid is publicly unavailable, we implement Pyramid with our framework.
- Weaviate [7] is an open-source vector database that employs a hash-based partitioning strategy to distribute data and maintains replicas across multiple nodes to ensure fault tolerance and high availability. It implements HNSW algorithm with fine-grained node-level locking for concurrency. We deploy Weaviate version 1.29.2 from the official Docker image.
- Milvus [13, 37] is a widely adopted vector database that implements a fixed-size partitioning strategy for indexing. Write operations are directed to a single active writable shard and converted into an immutable read-only segment until reaching its designated capacity. We deploy Milvus version 2.5.10 using the official Milvus-operator plugin on a Kubernetes cluster [4] .
- HAKES [15] is a disaggregated vector database that employs a two-stage ANN index architecture combining a filter stage and a refine stage. It utilizes machine learning techniques to fine-tune index parameters. Since the source code of HAKES is partially undisclosed, we implement HAKES based on our framework and use the training code of official repository.
- Harmony [41] is an IVF-style distributed vector search system that combines vector-based and dimension-based partitioning for load balance, and uses early-stop pruning on partial distances to further reduce computation and communication. We evaluate Harmony using its official open-source implementation.

It is worth noting that CoTra [51] is not included in the comparison: it relies on fine-grained inter-worker communication on an RDMA cluster, whereas TITAN targets commodity clusters with lightweight query routing and coarse-grained communication.

Evaluation Metrics. We evaluate the performance of TITAN using a standard set of metrics widely adopted in the ANN and distributed retrieval literature, with a primary focus on the trade-off between throughput and accuracy. Across all experiments, throughput is measured in **queries per second (QPS)**, representing the steady-state rate at which the system can process concurrent queries under saturated load. Accuracy is quantified using **Recall@k**, defined as the fraction of true nearest neighbors retrieved by the system.

Parameter Setting. We configure the number of buckets to be 1% of the dataset size, following the convention in IVF-based systems [3]. Both the meta-index and the sub-index employ HNSW graphs constructed with $ef_{\text{construction}} = 200$ and $M = 32$, which are standard defaults in hnswlib [28]. For overlapping data partitioning, based on preliminary experiments, we set $k = 10$, $\theta = 0.3$, and $n_{\max} = 2$ to balance replication overhead and routing efficiency. During query processing, the pruning retention rate λ is fixed at 0.8. Besides these, TITAN has three query-time parameters: meta-index search breadth $ef_{\text{search}}^{\text{meta}}$, sub-index search breadth $ef_{\text{search}}^{\text{sub}}$, and IVF selection threshold τ. To choose their values, we sample a small validation set, and measure the recall and QPS under different parameter combinations. For each target Recall@k, we select the combination that achieves the highest QPS on the validation set without dropping below the target, and then fix this combination

when running the full test set. Each point on the Recall–QPS trade-off curves therefore corresponds to a distinct, fixed parameter configuration tuned offline for that recall target.

Protocols. To ensure a fair and reproducible comparison, we follow the evaluation protocol established in prior large-scale ANN studies such as [34, 35]. For each baseline, we perform grid-search on the query-time parameters listed in the official manual or codebase (e.g. query-time search breadth ef_{search}, probing parameter n_{probe}), pick the configuration that maximizes QPS while meeting the target Recall@k. This policy ensures that each system is evaluated at its best-achievable performance under identical accuracy constraints, removing bias from inconsistent or suboptimal tuning.

Environment Setup. Unless otherwise noted, all experiments are conducted on a homogeneous cluster of 16 machines as worker nodes, plus one lightweight scheduler node. Each worker node is provisioned with identical hardware and software configurations. Each machine runs Ubuntu 24.04 LTS and is equipped with an Intel Xeon Platinum processor featuring 8 physical cores. CPU frequency scaling is disabled to avoid variation in per-core performance, and all machines are dedicated exclusively to the experiment during the evaluation period. The hardware configuration used for the billion-scale SIFT1B experiment is described separately in Section 6.2.

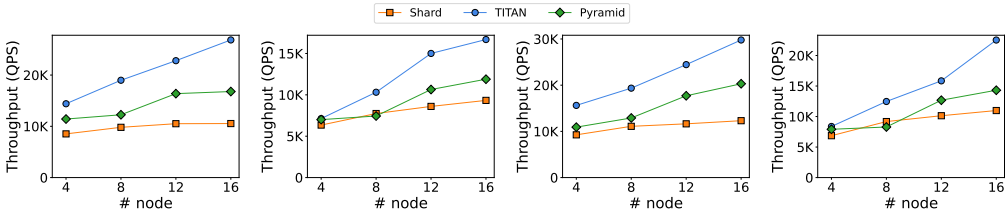
6.2 Main Results

End-to-End Query Performance Comparison. Figure 11 presents the throughput–accuracy curves of TITAN compared against six distributed ANNS baselines across five datasets at $k \in \{20, 100\}$. From the results, we observe the following two key findings.

Finding ❶: TITAN consistently achieves the highest throughput across all datasets and recall targets. Across all 10 configurations, TITAN delivers a 1.53×–4.52× speedup over the runner-up at Recall@k ≥ 0.95. This performance gain can be attributed to three design factors. First, the two-tier architecture routes each query to only 2–3 relevant workers by spatial locality, instead of broadcasting. Second, the topology-aware overlapping partitioning and angular bucket pruning further cut worker accesses and drop low-contribution buckets before local search. Third, the dual sub-index structure of each worker adaptively selects between proximity graph traversal and IVF scanning based on the selectivity of buckets, eliminating wasted computation. Together, these mechanisms allow TITAN to achieve superior recall-throughput trade-offs.

Finding ❷: Baselines lag due to architectural limitations. Broadcast systems (Shard HNSW, Milvus, Weaviate) can reach high recall, but routing every query to all (or most) workers caps throughput and scalability. Shard HNSW demonstrates competitive performance, achieving second or third throughput by leveraging HNSW's graph structure and lightweight query workflow. Pyramid shows moderate performance across all datasets through its similarity-based partitioning, but falls behind TITAN due to suboptimal data partitioning and the absence of fine-grained per-worker optimizations. HAKES exhibits dataset-dependent viability as it achieves competitive recall performance on LAION-T2I-20M but struggles to reach recall > 0.90 on the other datasets. This limitation stems from the IVF-PQ index structure, which suffers from severe quantization distortion in lower-dimensional spaces. Harmony reaches competitive recall on most datasets by increasing n_{probe} , but its IVF-style multi-granularity search must probe more lists and aggregate partial distances across nodes, yielding lower throughput than graph-based baselines. On LAION-T2I-20M dataset, it remains at extremely low recall even at large n_{probe}.

Throughput at Recall Target. To provide a direct comparison at a given quality target, Figure 12 reports the throughput achieved by each system at Recall@k=0.95. Note that HAKES fails to reach this recall target on any dataset and is therefore omitted from the comparison. The results



(a) SIFT10M @ k=20 (b) SIFT10M @ k=100 (c) Deep10M @ k=20 (d) Deep10M @ k=100

Fig. 13. The throughput scalability of TITAN and the baselines when varying the worker nodes at 0.95 recall target.

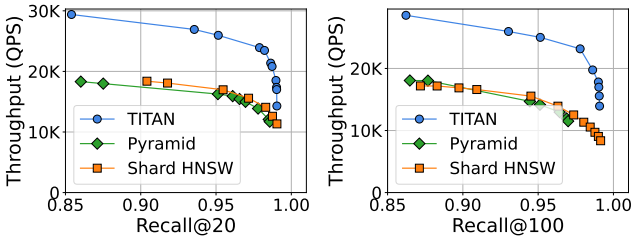


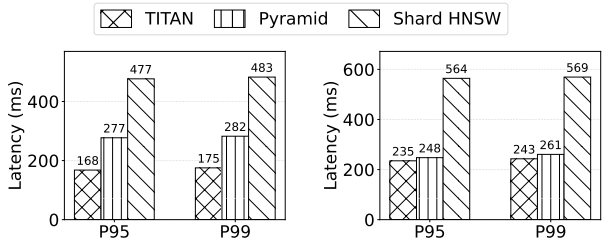
Fig. 14. Query recall-throughput curves on SIFT1B.

consistently demonstrate the superiority of TITAN, which achieves the highest throughput across all datasets. In all configurations, the average speedup of TITAN is 2.645×. In particular, on the SpaceV100M dataset at $k = 20$, TITAN delivers a maximum speedup of 4.52× over the runner-up method.

Billion-Scale Evaluation. To further evaluate the performance advantage of TITAN at the billion-vector scale, we conduct an additional experiment on the SIFT1B dataset. Due to the substantially higher memory and computational requirements, this experiment is conducted on a separate cluster of 16 worker machines, where each worker is equipped with an Intel Xeon Granite Rapids processor featuring 24 physical cores. Since billion-scale index construction incurs substantially higher preprocessing costs and requires prohibitively long construction times for several systems, we omit Weaviate, HAKES, Milvus, and Harmony. These baselines already underperform the remaining baselines in 100M-scale experiments. We therefore compare TITAN with Pyramid and Shard HNSW, the two strongest baselines in 100M-scale evaluation. As shown in Figure 14, TITAN consistently achieves the best Recall@k–QPS trade-off for $k \in \{20, 100\}$, demonstrating that its performance advantage persists on the billion-scale dataset.

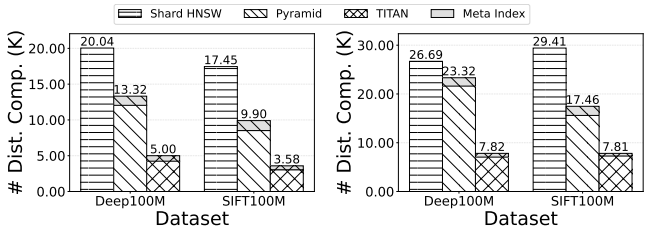
Scalability. We evaluate the scalability of TITAN on the SIFT10M and Deep10M datasets to understand how its performance evolves as the size of the cluster increases. As illustrated in Figure 13, we vary the number of workers while keeping all other settings fixed. We compare the throughput against the second and third best approaches Shard HNSW and Pyramid. Notably, they also represent uniform-partitioning-based and similarity-partitioning-based distributed ANNS approaches respectively.

Across all configurations, TITAN exhibits a strong near-linear scaling trend. When the number of workers increases, TITAN can effectively harness additional computational resources without introducing bottlenecks at either the scheduler or worker level. This behavior confirms that the



(a) Deep10M (b) SIFT10M

Fig. 15. P95 and P99 end-to-end latency at Recall@20 ≥ 0.95



(a) k=20 (b) k=100

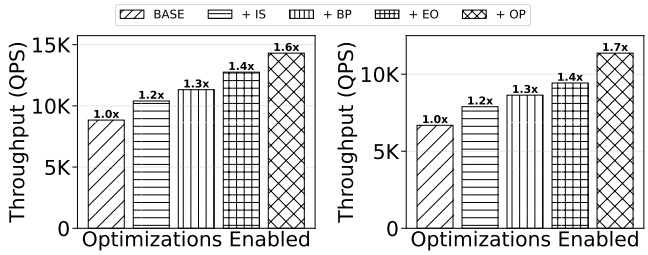
Fig. 16. The average number of distance computations per query of TITAN and the baselines at 0.95 recall target. The distance computations for the meta-index are also indicated.

hierarchical meta-index design successfully constrains queries to a small, predictable set of nodes, preventing the broadcast overhead commonly observed in uniform partitioning strategies. In contrast, both Pyramid and Shard HNSW exhibit noticeably worse scaling performance. Although Pyramid significantly outperforms the uniform-partitioning-based baseline with 12 and 16 nodes, its random sampling approach for the meta-index and graph partition-based query routing still significantly limits its scalability. For Shard HNSW, although increasing the number of workers reduces the size of each local graph, the search complexity decreases only logarithmically with respect to the graph size. Consequently, dividing the dataset into more partitions yields only marginal reductions in the per-worker search cost.

Latency Distribution. Beyond throughput, we report the end-to-end P95/P99 query latency under the same concurrent workload as it is important for distributed systems. For each baseline, we select the lowest P95/P99 result among those achieving Recall@20 ≥ 0.95. As shown in Figure 15, TITAN consistently attains the lowest tail latency on both datasets, which indicates that TITAN's throughput gains do not come at the cost of worse tail latency. As end-to-end latency is dominated by the slowest involved worker under concurrent workload, TITAN shortens this critical path by routing query to fewer workers through graph-aware overlapping partitioning and by reducing their search cost via local search optimizations.

Computational Efficiency. To better understand TITAN's performance advantage, we analyze the average number of distance computations per query for TITAN, Shard HNSW and Pyramid at a fixed recall target of 0.95 across two datasets at $k \in \{20, 100\}$.

As shown in Figure 16, TITAN consistently requires the fewest distance computations in all evaluated configurations. On the two datasets, TITAN reduces the number of distance computations



(a) Deep10M (b) SIFT10M

Fig. 17. Speedup over BASE at 0.90 Recall@20, BASE denotes the baseline TITAN without optimizations, IS represents the dual-index selection, BP denotes the bucket pruning, EO denotes the entry optimization, and OP indicates the graph-aware overlapping partitioning.

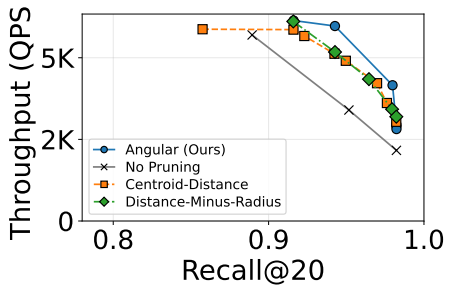
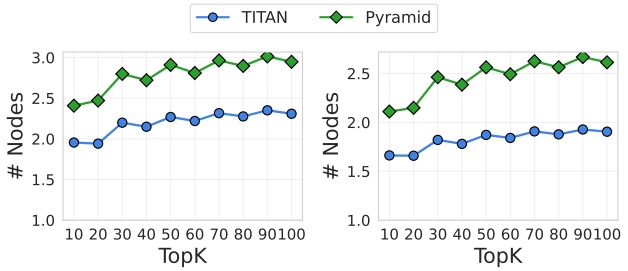


Fig. 18. Pruning strategy ablation on SIFT10M. *Angular* is our angular-based bucket pruning; *No Pruning* retains all buckets returned by meta-index routing; *Centroid-Distance* prunes buckets by query–centroid distance, and *Distance-Minus-Radius* prunes by $d(q, c) - r$ for each bucket.

by 70%–79% compared to Shard HNSW and by 55%–66% relative to Pyramid. Notably, the reduction is observed in both phases of the query pipeline. In particular, TITAN incurs significantly fewer distance computations during the meta-index search phase than Pyramid. These results confirm that TITAN's optimizations effectively minimize wasted computations while preserving high recall, resulting in higher QPS under resource constraints.

6.3 Micro Results

Ablation Study. We quantify the contribution of each optimization by incrementally applying TITAN's techniques: Index Selection (IS), Bucket Pruning (BP), Entry Optimization (EO), and Overlapping Partitioning (OP). As shown in Figure 17, a full set of optimizations yields an average 1.54× throughput speedup over the base architecture on the Deep10M and SIFT10M [22, 43] datasets at Recall@20 = 0.90. IS provides a 19% improvement by allowing each worker to select an optimal local index based on its bucket count. Adding BP further increases throughput by 9% on average through angular-based filtering that prunes distant buckets before sub-index search. EO contributes an additional 11% gain by reducing the distance of graph traversal through meta-index-guided entry points. Finally, OP improves performance by 17% through selectively replicating boundary buckets, thereby reducing node access. Overall, the ablation results demonstrate that each component of TITAN plays a critical role in the overall performance.



(a) SIFT100M (b) Deep100M

Fig. 19. Worker node access number under different top-k requirements at 0.95 recall target of TITAN and Pyramid.

To further validate the angular-based pruning, we compare it against no pruning, centroid-distance, and distance-minus-radius on the SIFT10M dataset, tuning each strategy within its own parameter space and reporting its best recall–throughput results. We did not include fixed top-k retention because varying the number of retained meta-index candidates (i.e., buckets) is already covered by sweeping $ef_{\text{search}}^{\text{meta}}$. As shown in Figure 18, three pruning strategies outperform no pruning baseline. At Recall@20≥0.95, angular pruning achieves about 1.2–1.4× higher QPS than the distance-based baselines and up to ~1.8× over no pruning strategy. The results demonstrate that solid-angle estimation preserves the recall–throughput trade-off under aggressive pruning.

Node Access. A key factor contributing to TITAN's high throughput is its ability to route queries to only a small subset of workers through meta-index-guided spatial partitioning, avoiding the broadcast overhead inherent in uniform-partitioning-based approaches. To quantify this routing efficiency, we measure the average number of workers accessed per query on the SIFT100M and Deep100M datasets of TITAN and Pyramid with a 16-worker cluster at 0.95 recall across varying top-k values, as shown in Figure 19. The results reveal two insights, as described below.

First, TITAN consistently accesses fewer workers than Pyramid in all k values, routing an average of fewer than 3 workers to achieve target recall. Second, the query routing is insensitive to k, indicating that increasing k extracts more candidates from already-accessed partitions rather than expanding the search scope, demonstrating the effectiveness of the meta-index in capturing spatial locality.

These results show that TITAN accesses fewer than 16% of workers under typical workloads while reaching the same Recall@k=0.95, demonstrating that our distributed architecture preserves spatial locality, and the overlapping partitioning further reduces node access by replicating boundary buckets, eliminating the need to query multiple partitions for neighbors near partition boundaries.

Memory Efficiency. We compare TITAN to Shard HNSW and Pyramid by computing the average memory usage per worker. As shown in Figure 20, Shard HNSW consistently achieves the lowest memory usage across all evaluated datasets as each worker only stores its own data. Pyramid incurs an average overhead of 1.1 GB over Shard HNSW due to the replicated routing index on each worker. TITAN introduces an additional 3.2 GB memory overhead on average, which can be attributed to two factors. First, TITAN stores more variables (e.g., bucket radii, bucket centroids and metadata in both indexes). Second, TITAN employs the overlapping partitioning to trade query efficiency by reducing worker access.

Notably, TITAN's meta-index incurs comparable memory usage to Pyramid's, yet achieves significantly higher efficiency. Consequently, when applying vector quantization technique to

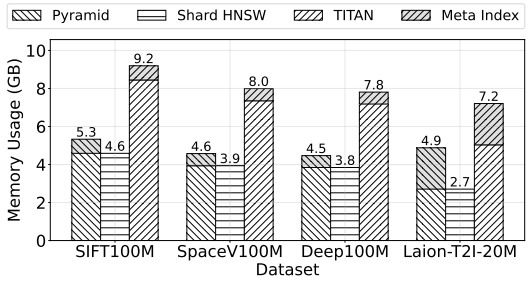


Fig. 20. The average memory usage on each worker node of TITAN and the baselines. The average memory usage of meta-index for TITAN and Pyramid are also indicated.

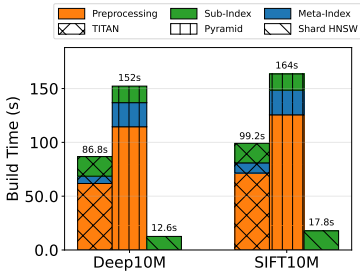


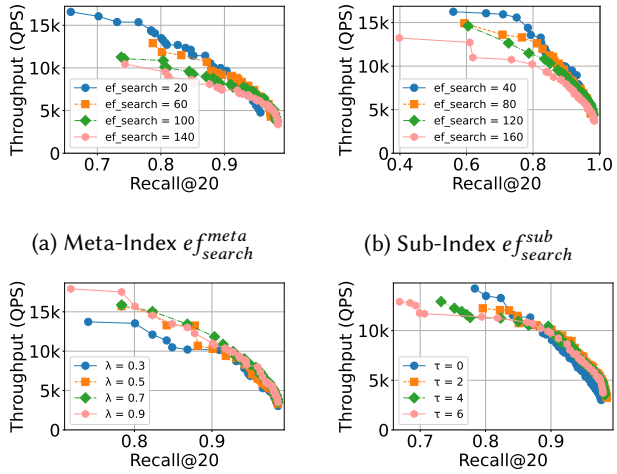
Fig. 21. Index construction time of TITAN, Pyramid, and Shard HNSW on 10M datasets.

the sub-index layer, TITAN is positioned to benefit more, since fewer nodes are accessed, the quantization-induced recall degradation affects fewer local searches, allowing aggressive compression without compromising recall. This presents a clear path to narrow the memory gap while maintaining superior throughput.

Index Construction Time. We compare the index construction time of TITAN with Pyramid, Shard HNSW under the 16-worker setting on two 10M datasets. As shown in Figure 21, TITAN is faster than Pyramid, while remaining slower than Shard HNSW as it only splits data and constructs per-worker proximity graphs. Pyramid's cost is dominated by preprocessing as it runs KaHIP algorithm to assign all vectors to partitions. In contrast, TITAN clusters vectors into buckets and partitions over centroids, thereby incurring lower preprocessing overhead. Although the sub-index construction takes slightly longer due to the overlapping partitioning, the overall construction time remains substantially cheaper than Pyramid.

Parameter Study. To provide practical guidance for parameter tuning and understand the robustness of TITAN, we evaluate the query-time parameters on Deep100M with 16 workers at k=20. We vary one parameter while fixing others to their default values and report the results in Figure 22. The parameters under study are: (a) meta index search breadth ef_{search}^{meta}, (b) sub-index search breadth ef_{search}^{sub} , (c) pruning retention rate λ and (d) IVF selection threshold τ. Note that the index-building parameters follow the setting in Section 6.1 and remain fixed across all datasets.

- *Meta-index* ef_{search}^{meta} exhibits minimal impact on performance as all configurations converge to a similar QPS at recall ≥ 0.95, as the meta-index size is small and a large search breadth is



(a) Meta-Index ef_{search}^{meta} (b) Sub-Index ef_{search}^{sub}

(c) Pruning Retention Rate λ (d) IVF Selection Threshold τ

Fig. 22. The query recall-throughput curves of TITAN under tuning of different query-time parameters.

unnecessary. We recommend $ef_{search}^{meta} \in [20, 60]$ as a practical default to balance routing accuracy with minimal overhead.

- *Sub-index* ef_{search}^{sub} shows a clear trade-off, as $ef_{search}^{sub} = 40$ achieves higher throughput but fails to exceed 0.90 recall, while larger values (e.g. $ef_{search}^{sub} = 120$) achieve up to 0.95 recall at the cost of reduced throughput to $\sim$ 4k QPS. This reflects the computational cost of graph traversal, as expanding the search breadth increases the distance computations and candidate evaluations on each worker. For recall > 0.95, we recommend $ef_{search}^{sub} \geq 100$. For recall > 0.90, $ef_{search}^{sub} \in [60, 80]$ provides better throughput.
- *Pruning retention rate* λ controls the aggressiveness of bucket pruning. From the results, $\lambda = 0.3$ shows significantly degraded performance, indicating overly aggressive pruning discards too many relevant buckets, and $\lambda = 0.7$ outperforms others at 0.90 recall. We recommend $\lambda = 0.7 \sim 0.9$ to effectively prune distant buckets while retaining sufficient candidates for recall targets.
- *IVF selection threshold* τ determines when to switch from graph traversal to IVF scanning based on the number of buckets assigned by the meta-index. At low to medium recall (0.75–0.85), using pure graph traversal always achieves the highest QPS due to its ability to explore beyond bucket boundaries, while higher τ values show slightly lower throughput in this regime due to the bucket-restricted nature of IVF scanning. At recall > 0.90, $\tau = 2$ and $\tau = 4$ achieve optimal throughput ($\sim$ 10,000 QPS at Recall@20 = 0.90), while $\tau = 0$ and $\tau = 6$ drop to $\sim$ 9,000 QPS. The root cause is that lower τ incurs unnecessary graph traversal overhead, while higher τ over-relies on IVF and misses relevant vectors. We recommend $\tau = 2$–4 as a practical default.

7 Related Work

Distributed ANNS. Distributed approximate nearest neighbor search has attracted growing attention as vector datasets scale beyond the capacity of single machines. Early systems such as Milvus [37] and Weaviate [40] rely on uniform partitioning, which simplifies data distribution but incurs high computational overhead. More recent systems, including Pyramid [6], HAKES [15], and Harmony [41], move toward similarity-aware partitioning. Pyramid constructs a coarse meta-index

via random sampling to route queries to a subset of nodes, whereas HAKES [15] adopts a two-stage partitioning-based index with learned compression parameters to improve storage efficiency and search performance. Harmony [41] further combines vector- and dimension-based partitioning and applies early-stop pruning on partial distances. CoTra [51] instead collaborates across workers with fine-grained RDMA communication to retain more from global graph pruning. TITAN advances this line by integrating a representative meta-index with adaptive dual sub-indexes, achieving routing precision without relying on expensive learned models or RDMA communication.

Data Partitioning Strategy of ANNS. Data partitioning is an important technique for both distributed ANNS and disk-based ANNS. It has also been studied in spatial settings, e.g., Lu et al. [26] partition data via Voronoi regions for kNN joins on MapReduce. However, high-dimensional ANNS requires co-designing partitioning with index structures and query routing due to the curse of dimensionality. Existing strategies of ANNS can be categorized into disjoint and overlapping partitioning. Overlapping partitioning has gained more attention as it reduces the number of queried partitions by replicating vectors. SPANN [5] replicates boundary vectors, but requires careful parameter tuning. SOGAIC [33] employs graph partitioning algorithms to minimize cross-partition edges, while Dong et al. [8] use reinforcement learning to optimize partition assignments. LIRA [46] also uses a learning-based method to predict relevant partitions. However, these methods either apply uniform replication or require expensive training. TITAN distinguishes itself by leveraging meta-index topology for selective boundary bucket replication, eliminating the need for costly training while reducing storage overhead without sacrificing recall.

Disk-based ANNS. Several works focus on handling large vector datasets using disk, and thereby reducing the demand on computational resources. DiskANN [19], PipeANN [12] build a small memory graph for navigation and retrieve results by searching an SSD-based index. However, their throughput is constrained by intensive disk I/Os. To address this issue, Starling [38] introduces the first data layout optimization on disk and significantly reduces the disk I/Os. MARGO [45] enhances the data layout of Starling by a parallel edge weight scoring algorithm and further improves the performance of graph reordering. Gorgeous [44] also targets this problem and improves performance by dividing the original data layout on SSD into index area and data area. While focusing on in-memory processing, TITAN's decoupled routing architecture and efficient node access patterns provide a foundation for future hybrid disk-memory systems that aim to minimize I/O bottlenecks.

8 Conclusion

In this paper, we present TITAN, a distributed system designed for high-throughput approximate nearest neighbor search (ANNS). TITAN employs a two-tier index architecture with a meta-index for global routing and a sub-index for local search, where the sub-index adaptively combines IVF and proximity graph indexes for efficient query processing. We incorporate several optimizations via co-design of data partitioning, query routing and local search to address the challenges of excessive node access and wasted computation on distributed vector search systems. Evaluations on large-scale benchmarks show that compared to state-of-the-art distributed ANNS methods, TITAN significantly improves the throughput of concurrent queries while maintaining high recall and delivers excellent scalability across various cluster configurations.

References

[1] Akari Asai, Sewon Min, Zexuan Zhong, and Danqi Chen. 2023. Retrieval-based Language Models and Applications. In *ACL*. 41–46.

[2] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2018. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. arXiv:1807.05614 https://arxiv.org/abs/1807.05614

[3] Dmitry Baranchuk, Artem Babenko, and Yury Malkov. 2018. Revisiting the inverted indices for billion-scale approximate nearest neighbors. In *ECCV*. 202–216.

[4] Eric A. Brewer. 2015. Kubernetes and the path to cloud native. In *SOCC*. 167.

[5] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: highly-efficient billion-scale approximate nearest neighbor search. In *NIPS*. 14 pages.

[6] Shiyuan Deng, Xiao Yan, K.W. Ng Kelvin, Chenyu Jiang, and James Cheng. 2019. Pyramid: A General Framework for Distributed Similarity Search on Large-scale Datasets. In *IEEE Big Data*. 1066–1071.

[7] Etienne Dilocker, Bob van Luijt, Byron Voorbach, Mohd Shukri Hasan, Abdel Rodriguez, Dirk Alexander Kulawiak, Marcin Antas, and Parker Duckworth. [n. d.]. *Weaviate*. https://github.com/weaviate/weaviate

[8] Yihe Dong, Piotr Indyk, Ilya Razenshteyn, and Tal Wagner. 2020. Learning Space Partitions for Nearest Neighbor Search. *ICLR* (2020).

[9] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graphs. *PVLDB* 12, 5 (2019), 461 – 474.

[10] Jianyang Gao and Cheng Long. 2024. Rabitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *SIGMOD* 2, 3 (2024), 1–27.

[11] Lars Gottesbüren, Laxman Dhulipala, Rajesh Jayaram, and Jakub Łącki. 2025. Unleashing Graph Partitioning for Large-Scale Nearest Neighbor Search. *PVLDB* 18, 6 (2025), 1649–1662.

[12] Hao Guo and Youyou Lu. 2025. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. In *OSDI*. 171–186.

[13] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, Zhenshan Cao, Yanliang Qiao, Ting Wang, Bo Tang, and Charles Xie. 2022. Manu: a cloud native vector database management system. *PVLDB* 15, 12 (2022), 3548–3561.

[14] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. In *ICML*. 10 pages.

[15] Guoyu Hu, Shaofeng Cai, Tien Tuan Anh Dinh, Zhongle Xie, Cong Yue, Gang Chen, and Beng Chin Ooi. 2025. HAKES: Scalable Vector Database for Embedding Search Service. *PVLDB* 18, 9 (2025), 3049–3062.

[16] Jui-Ting Huang, Ashish Sharma, Shuying Sun, Li Xia, David Zhang, Philip Pronin, Janani Padmanabhan, Giuseppe Ottaviano, and Linjun Yang. 2020. Embedding-based Retrieval in Facebook Search. In *SIGKDD*. 2553–2561.

[17] Qiang Huang, Jianlin Feng, Yikai Zhang, Qiong Fang, and Wilfred Ng. 2015. Query-aware locality-sensitive hashing for approximate nearest neighbor search. *PVLDB* 9, 1 (2015), 1–12.

[18] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *STOC*. 604–613.

[19] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in neural information processing Systems* 32 (2019).

[20] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.

[21] Herve Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57

[22] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in one billion vectors: Re-rank with source coding. In *ICASSP*. 861–864.

[23] Hui Li, Shiyuan Deng, Xiao Yan, Xiangyu Zhi, and James Cheng. 2025. SAQ: Pushing the Limits of Vector Quantization through Code Adjustment and Dimension Segmentation. *SIGMOD* 3, 6 (2025), 1–25.

[24] Dawei Liu, Bolong Zheng, Ziyang Yue, Fuhao Ruan, Xiaofang Zhou, and Christian S Jensen. 2025. Wolverine: Highly Efficient Monotonic Search Path Repair for Graph-Based ANN Index Updates. *PVLDB* 18, 7 (2025), 2268–2280.

[25] Haotian Liu, Kilho Son, Jianwei Yang, Ce Liu, Jianfeng Gao, Yong Jae Lee, and Chunyuan Li. 2023. Learning Customized Visual Models with Retrieval-Augmented Knowledge. In *CVPR*. 15148–15158.

[26] Wei Lu, Yanyan Shen, Su Chen, and Beng Chin Ooi. 2012. Efficient processing of k nearest neighbor joins using MapReduce. *PVLDB* 5, 10 (2012), 1016–1027.

[27] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68.

[28] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *TPAMI* 42, 4 (2020), 824–836.

[29] Microsoft. 2021. SpaceV1B. https://github.com/microsoft/SPTAG/tree/main/datasets/SPACEV1B.

[30] Priyanka Nigam, Yiwei Song, Vijai Mohan, Vihan Lakshman, Weitian (Allen) Ding, Ankit Shingavi, Choon Hui Teo, Hao Gu, and Bing Yin. 2019. Semantic Product Search. In *SIGKDD*. Association for Computing Machinery, 2876–2885.

[31] Peter Sanders and Christian Schulz. 2013. Think locally, act globally: Highly balanced graph partitioning. In *International Symposium on Experimental Algorithms*. Springer, 164–175.

[32] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, et al. 2022. Laion-5b: An open large-scale dataset for training next generation image-text models. *Advances in neural information processing systems* 35 (2022), 25278–25294.

[33] Yang Shi, Yiping Sun, Jiaolong Du, Xiaocheng Zhong, Zhiyong Wang, and Yao Hu. 2025. Scalable Overload-Aware Graph-Based Index Construction for 10-Billion-Scale Vector Similarity Search. In *WWW*. Association for Computing Machinery, 1303–1307.

[34] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamny, Gopal Srinivasa, et al. 2022. Results of the NeurIPS'21 challenge on billion-scale approximate nearest neighbor search. In *NeurIPS 2021 Competitions and Demonstrations Track*. PMLR, 177–189.

[35] Bing Tian, Haikun Liu, Yuhang Tang, Shihai Xiao, Zhuohui Duan, Xiaofei Liao, Hai Jin, Xuecang Zhang, Junhua Zhu, and Yu Zhang. 2025. Towards High-throughput and Low-latency Billion-scale Vector Search via CPU/GPU Collaborative Filtering and Re-ranking. In *FAST*. USENIX Association, 171–185.

[36] Karthik Venkatasubba, Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, and Jyothi Vedurada. 2025. BANG: Billion-Scale Approximate Nearest Neighbour Search Using a Single GPU. *IEEE Transactions on Big Data* 11, 6 (2025), 3142–3157.

[37] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *SIGMOD*. 2614–2627.

[38] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *SIGMOD* 2, 1 (2024).

[39] Xunguang Wang, Yiqun Lin, and Xiaomeng Li. 2023. CgAT: Center-Guided Adversarial Training for Deep Hashing-Based Retrieval. In *WWW*. 3268–3277.

[40] Weaviate. 2025. Weaviate. https://weaviate.io/

[41] Qian Xu, Feng Zhang, Chengxi Li, Lei Cao, Zheng Chen, Jidong Zhai, and Xiaoyong Du. 2025. Harmony: A scalable distributed vector database for high-throughput approximate nearest neighbor search. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–28.

[42] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, et al. 2023. Spfresh: Incremental in-place update for billion-scale vector search. In *SOSP*. 545–561.

[43] Yandex Billion-Scale Datasets. [n. d.]. https://research.yandex.com/datasets/biganns.

[44] Peiqi Yin, Xiao Yan, Qihui Zhou, Hui Li, Xiaolu Li, Lin Zhang, Meiling Wang, Xin Yao, and James Cheng. 2025. Gorgeous: Revisiting the Data Layout for Disk-Resident High-Dimensional Vector Search. arXiv:2508.15290

[45] Ziyang Yue, Bolong Zheng, Ling Xu, Kanru Xu, Shuhao Zhang, Yajuan Du, Yunjun Gao, Xiaofang Zhou, and Christian S. Jensen. 2025. Select Edges Wisely: Monotonic Path Aware Graph Layout Optimization for Disk-based ANN Search. *PVLDB* 18, 11 (2025), 4337–4349.

[46] Ximu Zeng, Liwei Deng, Penghao Chen, Xu Chen, Han Su, and Kai Zheng. 2025. LIRA: A Learning-based Query-aware Partition Framework for Large-scale ANN Search. In *WWW*. 2729–2741.

[47] Kai Zhang, Chongyang Tao, Tao Shen, Can Xu, Xiubo Geng, Binxing Jiao, and Daxin Jiang. 2023. LED: Lexicon-Enlightened Dense Retriever for Large-Scale Retrieval. In *WWW*. 3203–3213.

[48] Yanhao Zhang, Pan Pan, Yun Zheng, Kang Zhao, Yingya Zhang, Xiaofeng Ren, and Rong Jin. 2018. Visual Search at Alibaba. In *SIGKDD*. 993–1001.

[49] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. 2026. Retrieval-augmented generation for ai-generated content: A survey. *Data Science and Engineering* (2026), 1–29.

[50] Bolong Zheng, Ziyang Yue, Qi Hu, Xiaomeng Yi, Xiaofan Luan, Charles Xie, Xiaofang Zhou, and Christian S. Jensen. 2023. Learned Probing Cardinality Estimation for High-Dimensional Approximate NN Search. In *ICDE*. 3209–3221.

[51] Xiangyu Zhi, Meng Chen, Xiao Yan, Baotong Lu, Hui Li, Qianxi Zhang, Qi Chen, and James Cheng. 2026. CoTra: Towards Efficient and Scalable Distributed Vector Search with RDMA. *Proceedings of the ACM on Management of Data* 4, 1 (SIGMOD (2026), 1–27.

[52] Xiaoyao Zhong, Haotian Li, Jiabao Jin, Mingyu Yang, Deming Chu, Xiangyu Wang, Zhitao Shen, Wei Jia, George Gu, Yi Xie, Xuemin Lin, Heng Tao Shen, Jingkuan Song, and Peng Cheng. 2025. VSAG: An Optimized Search Framework for Graph-Based Approximate Nearest Neighbor Search. *PVLDB* 18, 12 (2025), 5017–5030.

Received 17 April 2026; revised 19 August 2026; accepted 12 September 2026